

University of Asia Pacific

Department of Computer Science and Engineering

CSE 450: VLSI Lab

LAB REPORT

Experiment Number: 03

Experiment Title: **Mini 8-bit Accumulator CPU**

Submitted by:

Name: **Israt Jahan Mou**

Student ID: **22201129**

Section: **C2**

Submitted to:

Md. Rasheduzzaman
Lecturer
Department of Computer Science and Engineering

Date of Submission: 23/10/26

1. Experiment Name

Design and Implementation of a Mini 8-bit Accumulator CPU in VHDL(set -9).

2. Objective

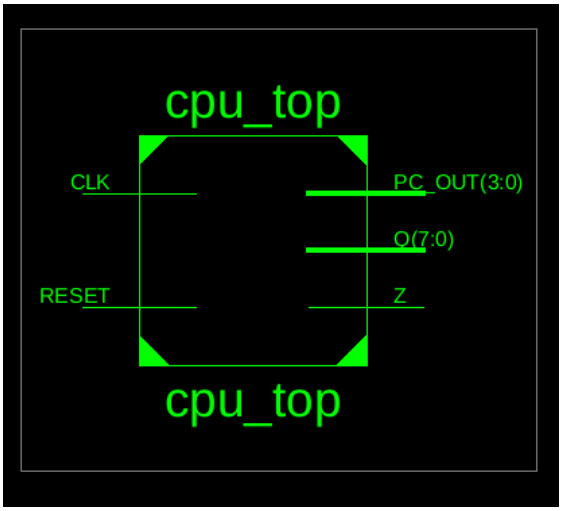
- To design a single-cycle 8-bit accumulator-based CPU using modular and hierarchical VHDL modeling.
- To implement the mandatory instructions `LDI`, `ADD`, and the assigned Set 9 instructions: `NOT`, `SHL`, and `JZ`.
- To integrate the datapath (ALU, ACC, PC, ROM) and control path using structural VHDL.
- To verify the CPU's cycle-by-cycle execution against the required program using a self-checking testbench.

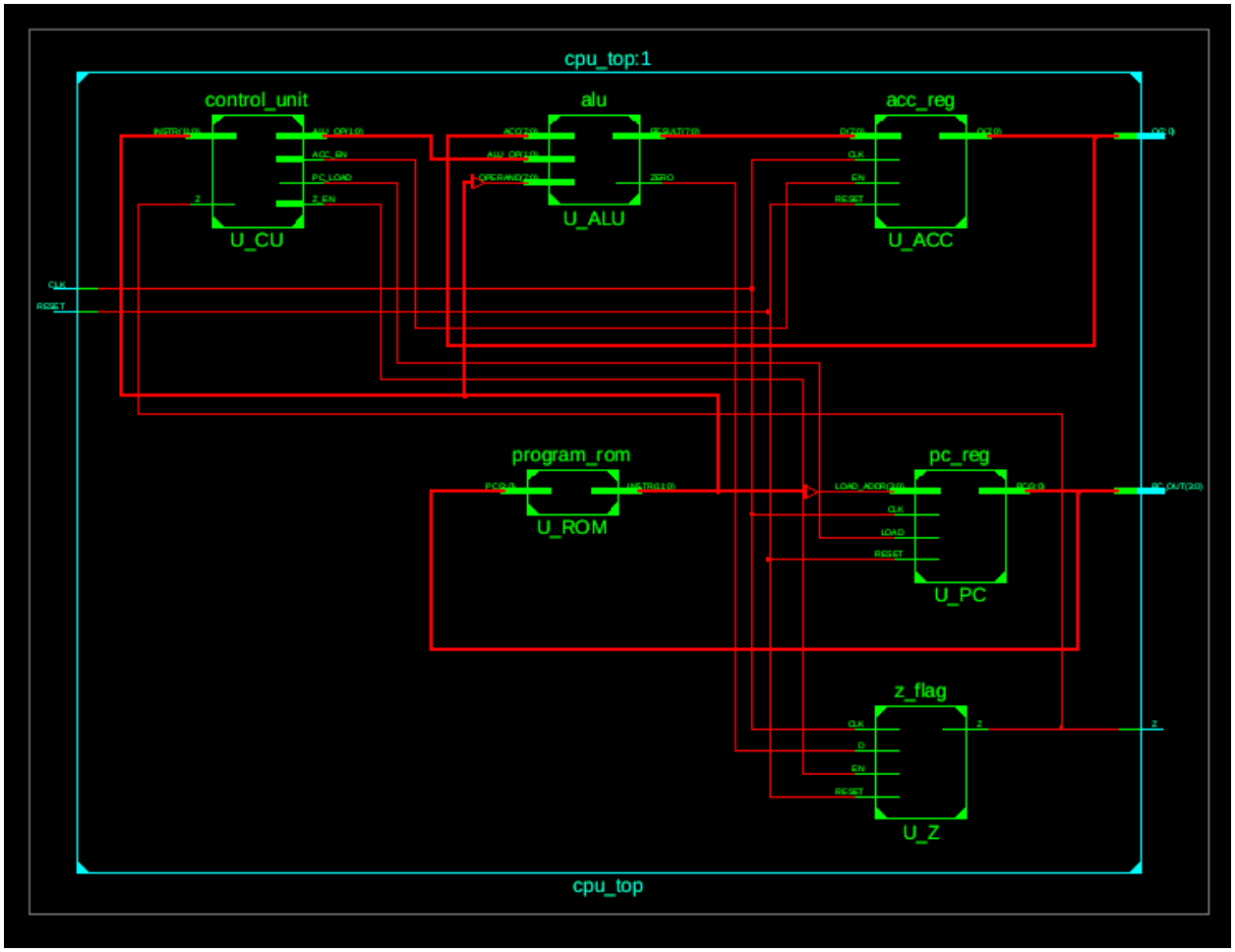
3. Apparatus / Hardware & Software Requirements

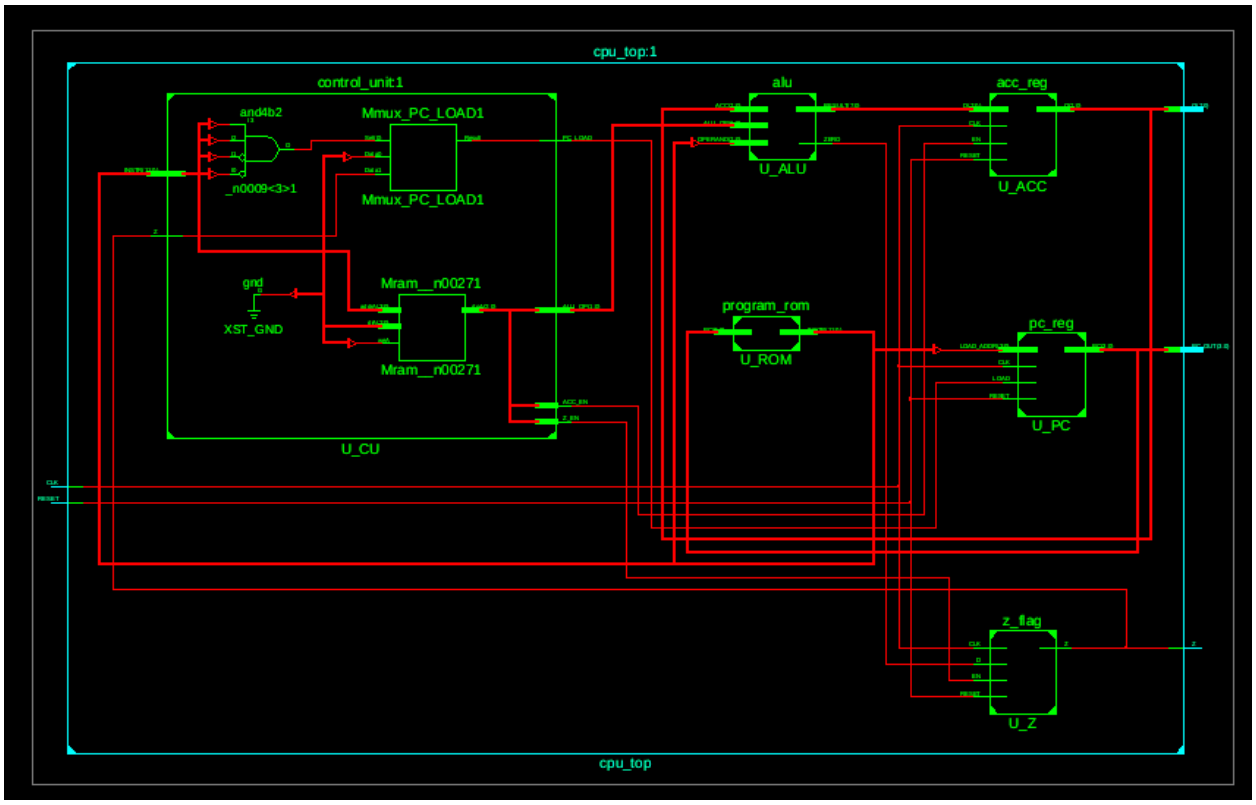
- Software: Xilinx ISE 14.7 (for synthesis and RTL schematic), ISim (for behavioral simulation).
- Language: VHDL (IEEE.STD_LOGIC_1164.ALL).
- Hardware: Standard PC.
- Design Blocks: Program Counter (PC), Program ROM, Control Unit, 8-bit ALU, Accumulator (ACC), and Zero Flag (Z). *(Note: The R register was omitted as Set 9 does not contain STA/LDR instructions).*

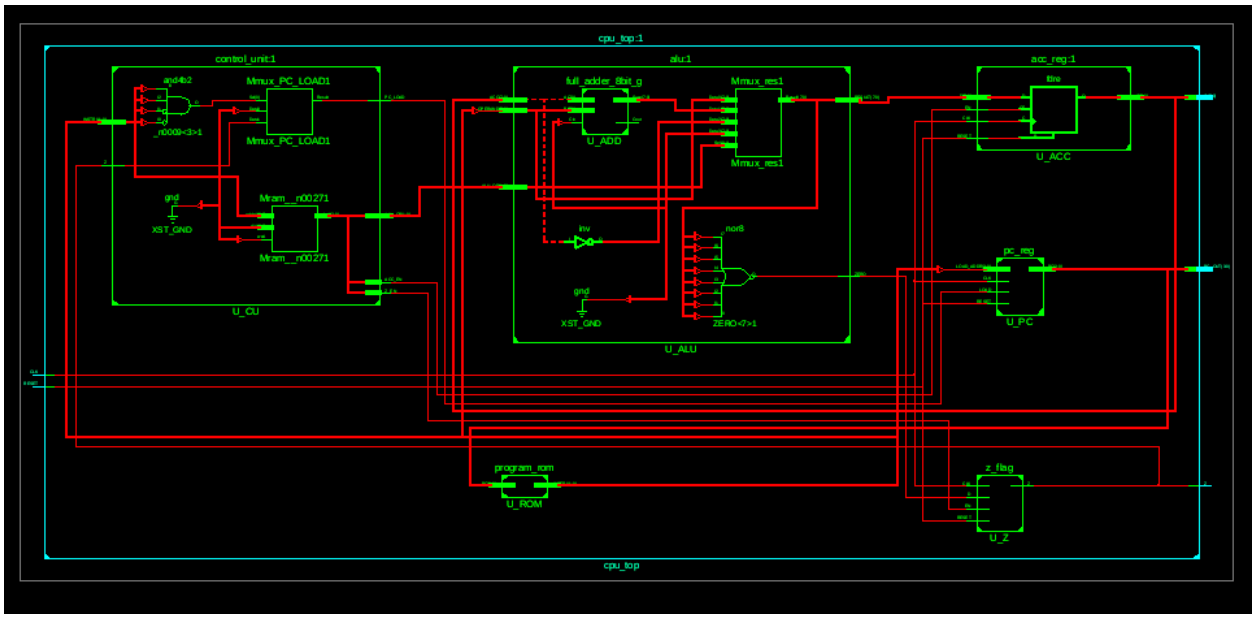
4. Circuit Diagram / Schematic

The CPU is built using a bottom-up structural approach. Primitive gates form the ALU, which is then connected to the Control Unit, ROM, and registers.




cpu_top:1
control_unit
alu
acc_reg
U_CU
U_ALU
U_ACC
CLK
RESET
program_rom
pc_reg
U_ROM
U_PC
z_flag
U_Z
cpu_top


cpu_top:1
control_unit:1
and4b2
Mmux_PC_LOAD1
_n0009<3>1
Mmux_PC_LOAD1
gnd
XST_GND
Mram__n00271
Mram__n00271
U_CU
alu
U_ALU
acc_reg
U_ACC
program_rom
U_ROM
pc_reg
U_PC
z_flag
U_Z
cpu_top


cpu_top:1
control_unit:1
and4b2
Mmux_PC_LOAD1
_n0009<3>1
Mmux_PC_LOAD1
gnd
XST_GND
Mram__n00271
Mram__n00271
U_CU
alu:1
full_adder_8bit_g
U_ADD
Mmux_res1
Mmux_res1
inv
gnd
XST_GND
nor8
ZERO<7>1
U_ALU
acc_reg:1
U_ACC
pc_reg
U_PC
program_rom
U_ROM
z_flag
U_Z
cpu_top

6. Output Analysis / Observations

The testbench applies a clock and a single-cycle active-high synchronous reset. The simulation successfully executed the Appendix B program for Set 9.

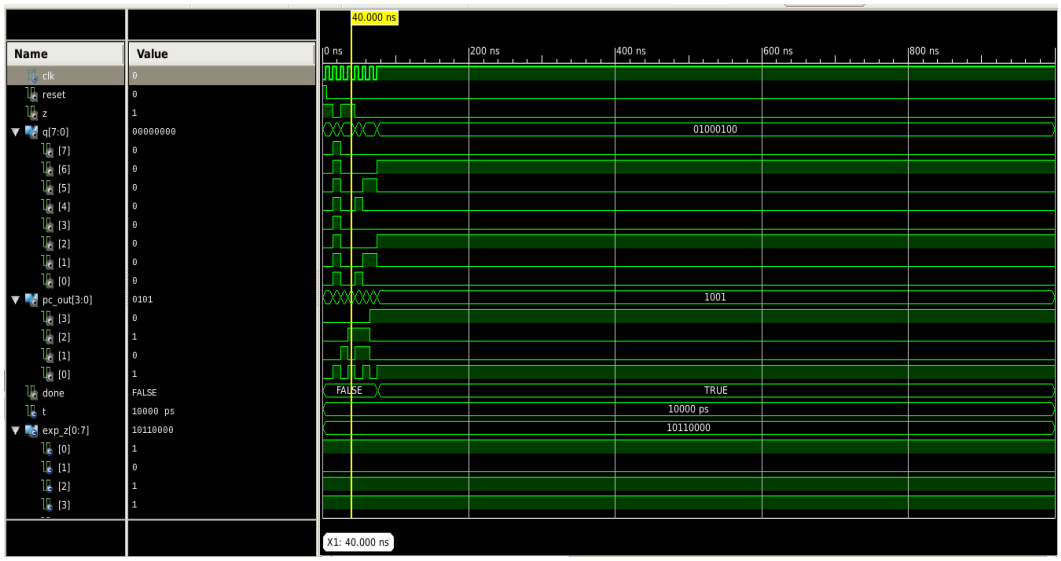
Expected vs. Actual Output (Set 9 Program):

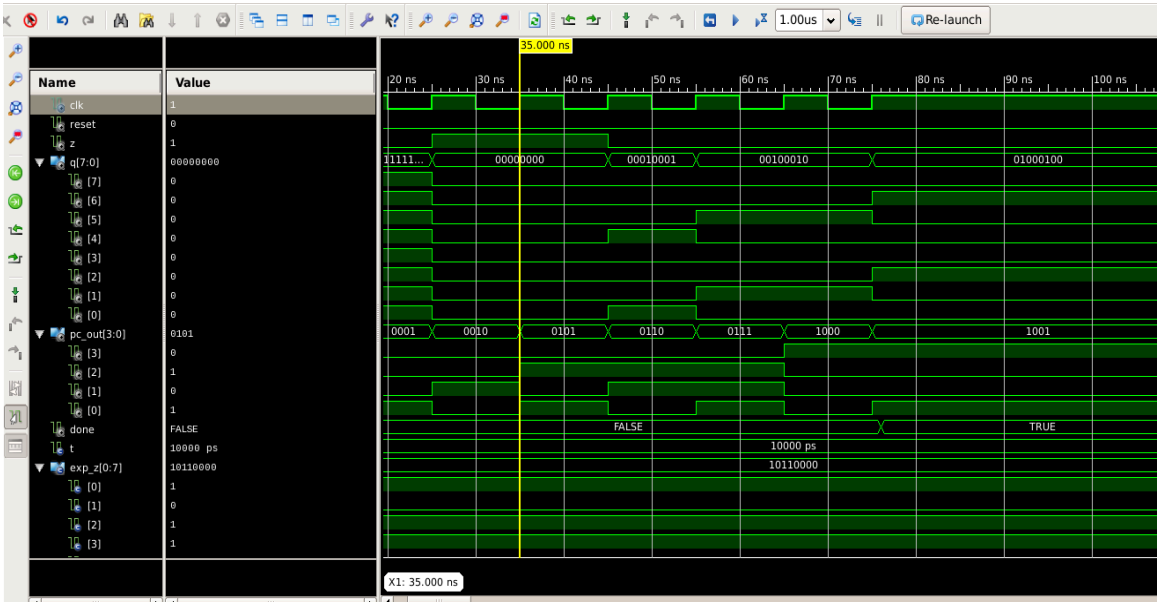
Cycle	PC	Instruction	Expected ACC	Actual ACC	Z Flag	Next PC
0	-	RESET = 1	0x00	0x00	1	0
1	0	LDI 0xFF	0xFF	0xFF	0	1
2	1	NOT	0x00	0x00	1	2
3	2	JZ 0x05 (Taken)	0x00	0x00	1	5
4	5	ADD 0x11	0x11	0x11	0	6
5	6	SHL	0x22	0x22	0	7
6	7	JZ 0x00 (Not Taken)	0x22	0x22	0	8
7	8	SHL	0x44	0x44	0	9

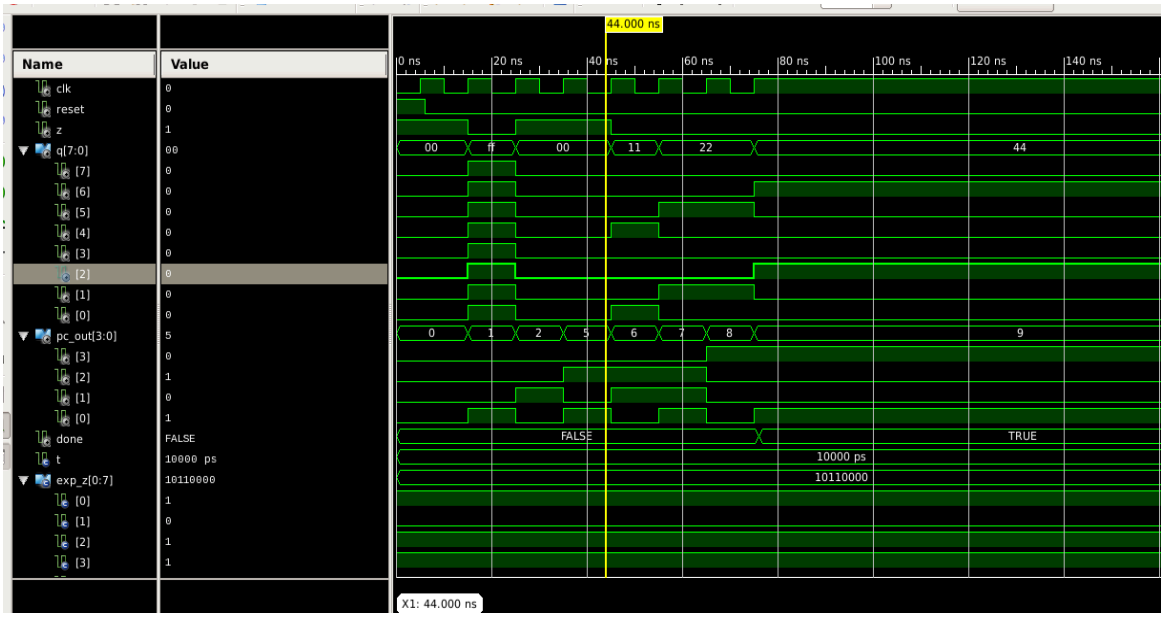
Key Observations:

- At Cycle 2, the NOT operation flips 0xFF to 0x00, which correctly sets the Zero flag (Z = 1).
- At Cycle 3, because Z = 1, the JZ 0x05 instruction correctly jumps the Program Counter directly to address 5, skipping instructions at addresses 3 and 4.
- At Cycle 6, Z = 0, so the JZ 0x00 instruction is not taken, and the PC increments normally to 8.

Finished circuit initialization process.
at 6 ns: Note: PASS cycle 0 ACC=0 Z='1' PC=0 (/cpu_tb/).
at 16 ns: Note: PASS cycle 1 ACC=255 Z='0' PC=1 (/cpu_tb/).
at 26 ns: Note: PASS cycle 2 ACC=0 Z='1' PC=2 (/cpu_tb/).
at 36 ns: Note: PASS cycle 3 ACC=0 Z='1' PC=5 (/cpu_tb/).
at 46 ns: Note: PASS cycle 4 ACC=17 Z='0' PC=6 (/cpu_tb/).
at 56 ns: Note: PASS cycle 5 ACC=34 Z='0' PC=7 (/cpu_tb/).
at 66 ns: Note: PASS cycle 6 ACC=34 Z='0' PC=8 (/cpu_tb/).
at 76 ns: Note: PASS cycle 7 ACC=68 Z='0' PC=9 (/cpu_tb/).
at 76 ns: Note: ALL 8 CYCLES MATCH APPENDIX B (SET 9) (/cpu_tb/).
ISim>
run all
ISim> **run all**







7. Result

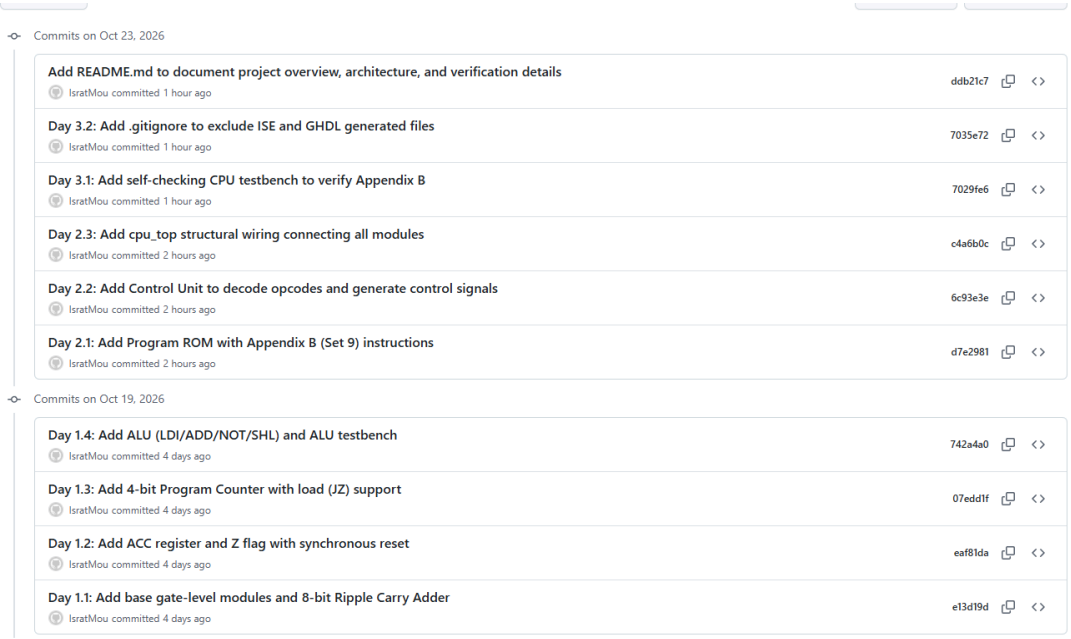
The 8-bit Accumulator CPU was successfully synthesized and simulated in VHDL. The self-checking testbench executed all 8 clock cycles and reported `ALL 8 CYCLES MATCH APPENDIX B (SET 9)` in the console. All assigned instructions functioned correctly.

8. Challenges Faced:

During the implementation, one of the main challenges was designing the CPU without using the built-in `+` operator for the `ADD` instruction. To strictly follow structural design principles, I reused the 8-bit Ripple Carry Adder (`full_adder_8bit_g`) built in previous labs, requiring careful port mapping inside the ALU. Another challenge was implementing the `JZ` (Jump if Zero) instruction. I had to ensure the Control Unit correctly checks the `Z` flag and generates the `PC_LOAD` signal at the exact clock cycle. Additionally, since Set 9 did not include `STA` or `LDR` instructions, I had to carefully analyze the architecture to realize the `R` (holding register) was not required, keeping the design optimized.

9. Github Repository Commits

Repository Link: https://github.com/IsratMou/Final_Project_CPU/commits/main/



10. Conclusion

This project provided valuable hands-on experience in designing a complete single-cycle processor. By adopting a modular design methodology, I learned how the control unit orchestrates the datapath using opcodes. Verifying the design with a self-checking testbench confirmed that the CPU correctly handles arithmetic, logic, and control flow (jump) instructions. In conclusion, this project bridged the gap between basic logic gates and complex sequential systems, demonstrating the fundamental principles behind modern computer architecture.